

System Architecture Overview

Part 1 — plain-language overview for stakeholders.

Part 2 — technical reference for engineers, architects, and reviewers.

An AI essay tutor that scores, explains, and coaches

The Writing Tutor is a web application that helps English-language learners improve their essay writing. A student is given a writing task, drafts an essay, and receives a structured, sentence-by-sentence analysis. The app then guides the student through corrections — in text or via a live voice tutor — and lets them resubmit to see their score improve.

The experience is organised as a clear four-step journey. The student always knows where they are, what they just did, and what comes next.

1 Brainstorm

[Plan & explore](#)

Before writing, the student talks through ideas with an AI brainstorming partner — by typing or by voice — to clarify their stance and collect supporting points for the prompt.

2 Write

[Draft](#)

The student writes their essay in a focused editor. An optional AI writing coach is available on the side to answer questions or unblock the student without writing the essay for them.

3 Review

[Learn & fix](#)

The essay is broken down sentence by sentence. Each sentence is shown alongside detected grammar issues, awkward phrasing, and suggested corrections. A voice tutor or a short quiz helps the student understand why each fix matters, not just what to change.

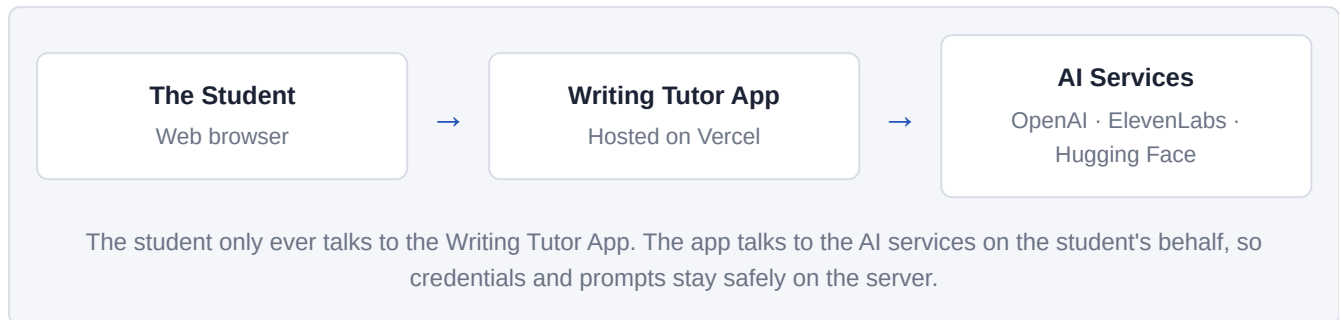
4 Score

[Measure progress](#)

The essay is scored against established writing criteria — content, grammar, vocabulary, cohesion, and more. After applying their fixes, the student can resubmit and see the side-by-side improvement in their score.

Three layers, one experience

The product is built from three layers that work together. Each layer has one clear job, which keeps the system reliable and makes it straightforward to evolve over time.



The three layers, explained

FRONT OF HOUSE

The interface

What the student sees and interacts with in their browser: the writing editor, the sentence-by-sentence review, the live voice tutor, and the scoring screen. Designed to feel calm and guided rather than busy.

THE KITCHEN

The application server

Receives the student's essay, decides which AI services to call, validates every response, and assembles the final result. This is also the only place that holds API keys — the browser never sees them.

THE SPECIALISTS

External AI services

Three specialised providers handle the parts that need machine intelligence: writing analysis, voice conversation, and impartial language scoring. The app coordinates them.

Who does what among the AI providers

OPENAI

The analyst & coach

Reads the essay, identifies grammar errors and awkward phrasing, generates explanations, runs the brainstorming and writing-help chats, and creates short practice quizzes.

ELEVENLABS

The voice tutor

Powers the natural-sounding spoken conversations in both the Brainstorm step and the sentence-by-sentence Review step, so the student can practise out loud.

HUGGING FACE

The impartial examiner

Provides two specialised language models that give a second, neutral opinion: one rates the writing across six dimensions, the other flags unnatural word choices.

Quality, safety, and the second opinion

Because the app makes judgements about a student's writing, every score and every correction needs to be defensible. Several design choices exist specifically to keep results trustworthy and consistent.

Every essay is reviewed by **two independent systems**: a generative model that explains what to fix, and a specialised scoring model that gives a stable, repeatable mark. The two are then combined into the score the student sees.

- **Cross-checked scoring**

The five language sub-scores (cohesion, syntax, vocabulary, phraseology, grammar, conventions) come from a specialised scoring model rather than from the generative AI alone. This makes scores stable: the same essay submitted twice receives the same mark.

- **Sentence-level accountability**

Feedback is anchored to individual sentences, not just paragraphs. The student can see exactly which sentence each suggestion refers to and decide whether to accept it.

- **Validated AI responses**

Every reply from an AI service is checked against a strict schema before it reaches the student. Anything malformed is rejected on the server instead of being shown as a broken screen.

- **Graceful fallbacks**

If the specialised scoring model is unavailable, the app falls back to the generative model and clearly marks the source of the score. The student is never blocked.

- **Credentials stay on the server**

API keys for the AI providers are held only on the application server. The student's browser never holds a key and cannot be used to run up costs or abuse the providers.

- **No essay storage by default**

The app is stateless with respect to student work — essays live in the current browser session and are passed to AI providers only for the duration of a single request.

Built on modern, well-supported foundations

The application uses widely adopted, actively maintained technology with strong ecosystems and developer availability. This protects the product against being stuck with niche or aging tools.

The web platform

The app is built on **Next.js** and **React** — the same foundation used by major publishers and SaaS products. It loads quickly, works on phones and laptops, and is easy to update.

How it is hosted

Deployed on **Vercel**, a managed hosting platform that scales automatically with traffic. No server administration is required, and updates roll out in minutes whenever code is published.

Design system

The visual language uses a consistent token-based design system. This means colours, spacing, and typography are defined in one place and applied uniformly, which keeps the product looking polished.

Operations

AI provider costs are usage-based and fully observable. The app logs each analysis run with a unique ID, so any reported issue can be traced end-to-end without re-running the student's essay.

Where the architecture is heading

The current shape supports the immediate product goal: a single student, a single essay, a complete coaching loop. A few natural extensions are already supported by the foundations:

- **Student accounts and history**

Adding sign-in and persistent essay history is a straightforward addition that does not change the core flow.

- **Teacher and classroom views**

The same scoring and review pipeline can feed dashboards for teachers tracking class progress over time.

- **Additional languages and task types**

The prompt-driven design means new writing tasks (letters, reports, academic essays) and additional target languages can be added incrementally.

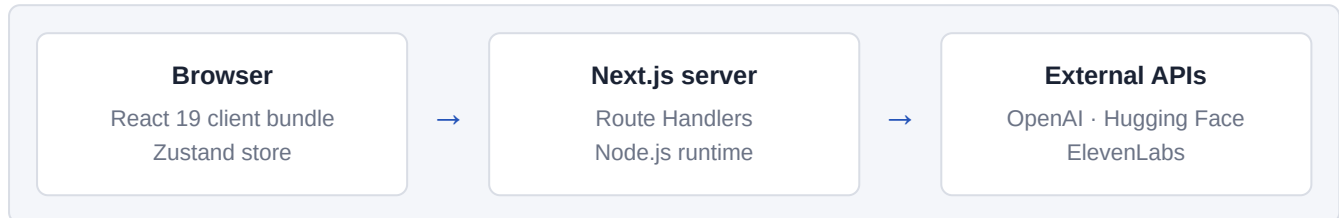
- **Swappable AI providers**

Each AI service sits behind a clear boundary, so any one provider can be replaced or supplemented without rewriting the rest of the system.

Processes, boundaries, and external services

The application is a single Next.js 16 deployment (App Router, React 19, Turbopack, `output: 'standalone'`) hosted on Vercel. There is no custom server, no database, and no persistent state — the system is a thin orchestrator that fans requests out to three external providers and returns a validated, composed response.

PROCESS TOPOLOGY



ROUTE HANDLERS (ALL IN `SRC/APP/API/*`)

ENDPOINT	PURPOSE	UPSTREAM	BUDGET
POST <code>/api/analyze</code>	Score essay; per-sentence grammar & phraseology enrichment	OpenAI chat · HF KevSun TEI · HF roberta-base fill-mask	maxDuration 60s 4 000 tokens
POST <code>/api/brainstorm-chat</code>	Multi-turn chat for the Brainstorm step (text mode)	OpenAI chat	400 tokens
POST <code>/api/write-tutor</code>	Side-panel coaching while the student drafts	OpenAI chat	600 tokens
POST <code>/api/review-tutor</code>	Per-sentence tutor reply (text mode), grounded in edits + flags	OpenAI chat	400 tokens
POST <code>/api/quiz</code>	Generates 2-stage MC + gap-fill quiz for a flagged sentence	OpenAI chat (JSON mode)	1 500 tokens

EXTERNAL SERVICES & INTEGRATION MODE

SERVICE	USED FOR	TRANSPORT	AUTH
OpenAI Chat Completions	All generative work (analyzer, chats, quiz)	openai SDK, server-side	OPENAI_API_KEY
HF Inference Endpoint (KevSun roberta-large)	Deterministic 6-dim essay scoring (TEI <code>raw_scores: true</code>)	fetch, server-side	HUGGINGFACE_API_KEY
HF Inference Router (FacebookAI/roberta-base)	Per-token <code><mask></code> rank check for phraseology flags	fetch, server-side	HUGGINGFACE_API_KEY
ElevenLabs Conversational AI	Live voice tutors (Brainstorm & per-sentence Review)	@elevenlabs/react, browser WebRTC	Public agent ID; no server key

All Route Handlers declare `runtime = 'nodejs'` and `dynamic = 'force-dynamic'`; nothing in the application is cached, prerendered, or revalidated. Middleware (`src/proxy.ts`) is a no-op pass-through that excludes `/api`, `/_next`, `/_vercel`, and static files.

POST `/api/analyze` — end-to-end

The analyze route is the central piece of the system: it composes scores from two independent models and enriches every sentence with masked-LM phraseology data. The same handler serves both the initial submission and later resubmissions.

01	Parse JSON body; reject non-JSON with 400.
02	Validate against <code>requestSchema = { text: 1-10 000, prompt: 1-2 000 }</code> . Extra fields are stripped.
03	Mint a request ID via <code>wtRunId()</code> ; log essay fingerprint (length + first/last chars) for traceability.
04	Fan out in parallel via <code>Promise.all</code> : <code>openai.chat.completions.create</code> with <code>response_format: { type: 'json_object' }</code> using <code>buildAnalyzerPrompt</code> , and <code>scoreKevSun(text)</code> against the HF endpoint.
05	KevSun postprocess : raw regression logits $\rightarrow 2.25 \cdot x - 1.25$, clamp to <code>[1, 5]</code> , snap to <code>0.5</code> . Returns <code>null</code> if endpoint URL is unset, response is non-200, response shape is wrong, or values are softmaxed (sum ≈ 1).
06	Strip code fences from the OpenAI message, <code>JSON.parse</code> , then validate against <code>analysisResultSchema</code> (Zod). Failure $\rightarrow$ 500.
07	Per-sentence enrichment (serial across sentences, parallel within): for each sentence call <code>checkPhraseology(original, grammarTokens)</code> . Each candidate word becomes a masked sentence sent to <code>roberta-base</code> with <code>top_k=100</code> .
08	Flag rule : a word is flagged when its rank in the MLM distribution at that position is > 80 , or it doesn't appear in the top 100. Stop-words and tokens already covered by a grammar edit are skipped.
09	Phraseology sentence score = $\max(1, 5 - \text{flags} \cdot 0.5)$, snapped to <code>0.5</code> .
10	Score composition (when KevSun is present): five linguistic dims come from KevSun, content from the LLM, then PTE recomposition.
11	Overall = <code>round05((content + cohesion + grammar + (syntax+phraseology)/2 + vocabulary + conventions) / 6)</code> .
12	Respond with the validated payload plus a <code>__debug</code> envelope containing the request ID, scoring source, raw KevSun, LLM scores, computed scores, and phraseology flag totals.

UI OVERALL (PTE 10-90 SCALE)

```
// src/modules/writing-flow/lib/score-mapping.ts
const toPteOverall = (x1to5: number): number =>
  clamp(10 + Math.round(((x1to5 - 1) / 4) * 80), 10, 90);
```

07 · MODULE LAYOUT & BOUNDARIES

One feature module, strict file-placement rules

All product code lives inside a single feature module `src/modules/writing-flow/`. The App Router shell is a single page that mounts `<AppShell />`. The folder layout below is enforced by project rules and reviewed on every change.

```
src/
  app/
    api/
      analyze/route.ts      # orchestrator: OpenAI + KevSun + phraseology
      brainstorm-chat/route.ts
      quiz/route.ts
      review-tutor/route.ts
      write-tutor/route.ts
    globals.css             # Tailwind v4 + @theme tokens (Pearson palette)
    layout.tsx              # root layout, no providers
    page.tsx                # <AppShell />
  lib/
    env.ts                  # @t3-oss/env-nextjs schema
    openai.ts               # singleton SDK client (server-only)
  modules/writing-flow/
    components/              # 35 React components (screens, cards, panels)
    constants/              # analyzer-prompt, steps, brainstorm, dev
    hooks/                  # use-analyze-essay, use-resubmit-essay,
                           # use-quiz-generation, use-*-voice, use-*-tutor
    lib/                    # pure helpers: score-kevsun, score-mapping,
                           # check-phraseology, assemble-edited-essay,
                           # sentence-fix, derive-paragraph-tabs,
                           # *-system-prompt, debug-log
    schemas/                # Zod: analysis, quiz, brainstorm-chat,
                           # review-tutor, write-tutor
    stores/use-flow-store.ts # single Zustand store (state + actions)
    types/                  # analysis.types, flow.types, quiz.types, review.types
    index.ts                # barrel: exports <AppShell /> and useFlowStore
    proxy.ts                # Next 16 middleware (pass-through)
```

NAMING & PLACEMENT RULES

KIND	CONVENTION	REQUIRED LOCATION
Component	PascalCase <code>.tsx</code>	<code>components/</code>
Hook	<code>useX</code> camelCase	<code>hooks/</code>
Zustand store	<code>use-x-store.ts</code>	<code>stores/</code>
Zod schema	<code>x.schema.ts</code>	<code>schemas/</code>
Pure utility	kebab-case	<code>lib/</code>

SERVER / CLIENT BOUNDARY

- Server-only modules.

`src/lib/openai.ts`, `score-kevsun.ts`, and `check-phraseology.ts` start with `import 'server-only'`.

- Client components.

The entire `writing-flow` UI tree, the Zustand store, and all hooks begin with `'use client'`.

- One feature, one store.

A single `useFlowStore` holds the whole UI state machine; selectors live in the same file.

08 · DATA MODEL

The contracts crossing every boundary

Every request and every response is validated with Zod on the server, and the analyze response is validated again on the client before it enters the store. The shapes below are the authoritative contract.

ANALYSIS RESULT — THE ANALYZE RESPONSE

```
type AnalysisResult = {
  scores: {
    cohesion: number; syntax: number; vocabulary: number;
    phraseology: number; grammar: number; conventions: number;
    content: number; overall: number;           // 1.0-5.0, step 0.5
  };
  sentences: AnalyzedSentence[];
  paragraphs?: { label: 'Introduction' | 'Conclusion' | `Body ${number}`;
    sentenceIndices: number[] }[];
  raw_kevsun?: RawKevSun;                       // raw model logits (debug/anchor)
  kevsun_anchor?: { min: number; max: number };
};

type AnalyzedSentence = {
  original: string;
  corrected: string;
  has_errors: boolean;
  grammar_edits: { type: 'replace' | 'delete' | 'insert';
    token: string; correction?: string }[];
  phraseology_flags: { token: string; rank: number;
    suggestions: string[] }[];
  phraseology_score?: number;                   // 1.0-5.0, server-computed
};
```

ZUSTAND STORE — USEFLOWSTORE (SINGLE STORE, ALL UI STATE)

```
type FlowState = {
  step: FlowStep; maxStep: FlowStep;           // nav state, forward-only
  prompt: string; essay: string;
  brainstorm: { mode: 'type' | 'voice'; messages: ChatMessage[]; isVoiceActive: boolean };
  analysis: AnalysisResult | null;             // current view
  initialAnalysis: AnalysisResult | null;       // preserved for comparison
  resubmitAnalysis: AnalysisResult | null;
  edits: Record<number, string>;               // sentenceIndex → corrected text
  selectedSentenceIndex: number | null; tutorMode: 'type' | 'voice';
  quiz: QuizState;                             // MC + gap-fill state machine
  editMode: boolean; editingSentenceIndex: number | null;
  fixedSentenceIndices: number[]; reviewMessages: ReviewChatMessage[];
  runLog: RunLogEntry[];                      // every analyze/resubmit round-trip
};
```

RUN LOG ENTRY — ONE PER ANALYZE ROUND-TRIP

```
type RunLogEntry = {
  runId: string;
  kind: 'initial' | 'resubmit';
  at: string;                                   // ISO timestamp
  textLength: number;
  editCount: number;
  scoringSource: 'kevsun' | 'llm-fallback' | 'unknown';
  scores: EssayScores;
  overallPte: number;                          // 10-90 display scale
};
```

How it runs in production

ENVIRONMENT VARIABLES — VALIDATED IN `SRC/LIB/ENV.TS`

VARIABLE	SCOPE	REQUIRED	DEFAULT / NOTES
<code>OPENAI_API_KEY</code>	server	yes	Model defaults to <code>gpt-5.4-mini</code> (override via <code>OPENAI_MODEL</code>)
<code>HUGGINGFACE_API_KEY</code>	server	yes	Used for both HF calls
<code>HUGGINGFACE_FILL_MASK_URL</code>	server	no	HF roberta-base router (default)
<code>HUGGINGFACE_KEVSUN_ENDPOINT_URL</code>	server	optional	Unset → LLM fallback, run tagged <code>llm-fallback</code>
<code>NEXT_PUBLIC_ELEVENLABS_AGENT_ID</code>	client	yes	Review (per-sentence) voice tutor
<code>NEXT_PUBLIC_ELEVENLABS_BRAINSTORM_AGENT_ID</code>	client	optional	Brainstorm voice tutor (separate agent)

SECURITY BOUNDARIES

- **Server-only secrets.**

`OPENAI_API_KEY` and `HUGGINGFACE_API_KEY` are declared under `server:` in the env schema, so any client import would be a build-time error. The OpenAI SDK and HF `fetch` calls live in `'server-only'` modules.

- **Zod-validated in and out.**

Every `POST` body is validated server-side (limits: `text ≤ 10 000`, `prompt ≤ 2 000`, `messages ≤ 50`); analyze responses are revalidated client-side against `analysisResultSchema` before they enter the store.

- **ElevenLabs key never reaches the browser.**

Voice tutors use the ElevenLabs public-agent flow with dynamic variables; no `ELEVENLABS_API_KEY` exists in the app.

OBSERVABILITY

- **Per-run identifiers.**

Every analyze call mints `wtRunId()` and threads it through logs and the `__debug` envelope returned to the client, so a single submission is traceable end-to-end.

- **Scoring source surfaced.**

Each run is tagged `kevsun`, `llm-fallback`, or `unknown`; the tag is shown on the Score screen and stored in `runLog` together with an essay fingerprint (length + head/tail chars) instead of the raw text.

DEPLOYMENT & KNOWN LIMITATIONS

`output: 'standalone'` on Vercel; same artefact runs in any container. No workers, queues, or database. Horizontal scale is bounded by upstream quotas and the analyze route's `maxDuration: 60`.

- **No persistence.**

All state lives in memory in the browser session. Refresh resets the flow. Adding auth and persistent essay history is the most natural next step and does not require changes to the analyze pipeline.

- **Pipeline shape, by design and by TODO.**

Phraseology runs one sentence at a time (parallel within) to keep HF concurrency under the 60s route ceiling; `use-analyze-essay` and `use-resubmit-essay` are hand-rolled `fetch + useState` with in-file TODOs to move to `TanStack useMutation`.